CS 690 P3

Thomas Copas, Liam Nicholson, Yuang Zhang, Yanlin Zhou

April 2025

1 Introduction

In this project, we constructed a pipeline file (MP3.ipynb) to record RGB (sensor.camera.rgb in Carla), depth (sensor.camera.depth), and LIDAR (sensor.lidar.ray_cast) sensor data from the Carla simulator. This sensor data is then reconstructed as an aligned point cloud, producing frame-by-frame .ply files as well as .avi files for both RGB and depth data as final output.

2 Technical Details

When capturing our baseline truth measurements, we put a LiDAR sensor one meter above the car. To fill in the point data and achieve 360° coverage, we attached multiple RGBD cameras around the car: one at each headlight, one at the rear, and one facing each corner of the car. The RGBD data was saved as individual .png files for later use, as well as .avi files for video analysis.

To align the RGBD and LiDAR data, all files were checked for common frame numbers, skipping where there was insufficient data from a sensor. Then, for each frame, all LiDAR points were checked to see which camera could visualize it best. This process involved computing and applying the rotation and translation matrices between the LiDAR sensor and each camera. Projecting the LiDAR point to the camera frame allowed us to perform depth checks, selecting the camera with the closest approximation, then sample the color at the stored (u, v) position. Once this is complete, the aligned point cloud is saved per frame as [frame number].ply.

3 How To Use Our File

Open the Carla simulator to port 2000. Once it is ready, run MP3.ipynb. The car in the simulator will be spawned, and a separate window will open with the car going in a straight line. Once you've gathered the data you wish to inspect, press the 'q' key to end the data gathering phase. MP3.ipynb will then generate output.ply files in an "aligned" directory, with each file within that directory denoted by its frame. This process may take some time depending on the amount of data gathered and how powerful your hardware is, so please be patient if you wish to see the full output, but you can freely inspect the output .ply files as they are generated. If a frame by frame visualization is wanted for the RGBD unprojection and lidar coloring, set VISUALIZING = True in the Pipeline section. If a color map is wanted for which LiDAR points are detected by which camera, set CAMERA_COLORING = TRUE. Note this is created in a clockwise fashion starting with the left headlight RGBD data, thus the rightmost camera that can visualize a point will take priority. If occlusion is wanted, set USE_OCCLUSION = True.

4 Results

Utilizing the VISUALIZING boolean, which creates an Open3D window, the following images can be observed:

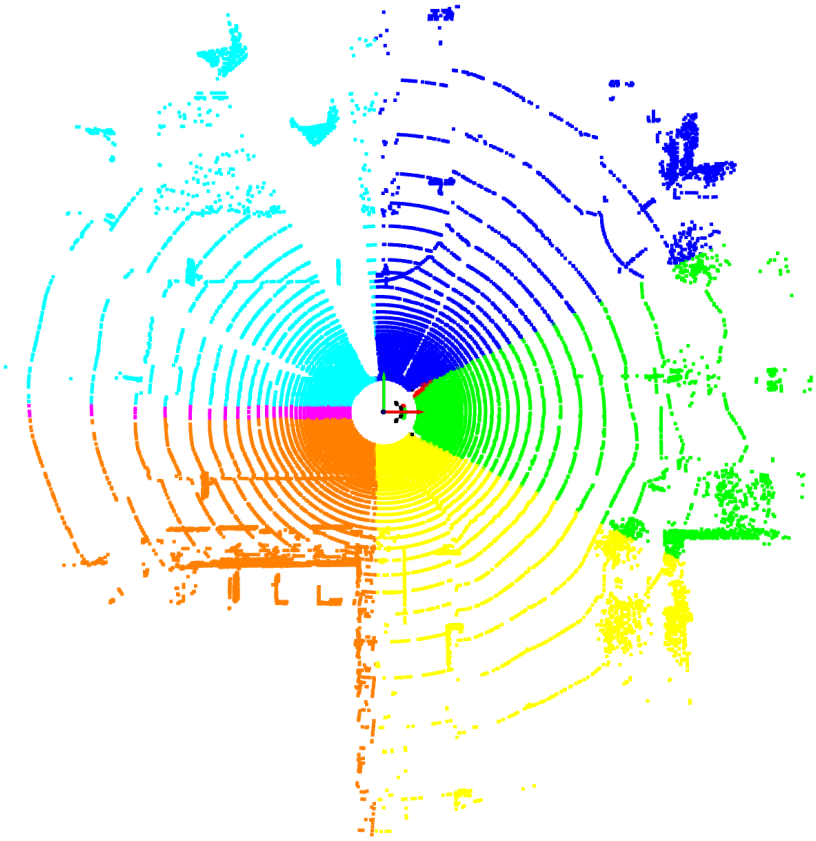

Figure 1: CAMERA_COLORING=True, USE_OCCLUSION=False

In Figure 1, each pixel is mapped to the rightmost camera that can see it. Since the left headlight is first in the clockwise rotation, the left front camera in dark blue steals all the points that would otherwise be red.

Figure 2: CAMERA_COLORING=False, USE_OCCLUSION=False

After switching to RGB, the visualization shows that all meaningful LiDAR points are filled in, with the few unaligned points (in red) coming from angles that would require the camera to be far from the car looking toward the front. It is worth noting that the points a bit above the green axis arrow are incorrectly a light blue. This is because the points are being forcefully sampled from the back left camera since it comes clockwise before in comparison to the front left.

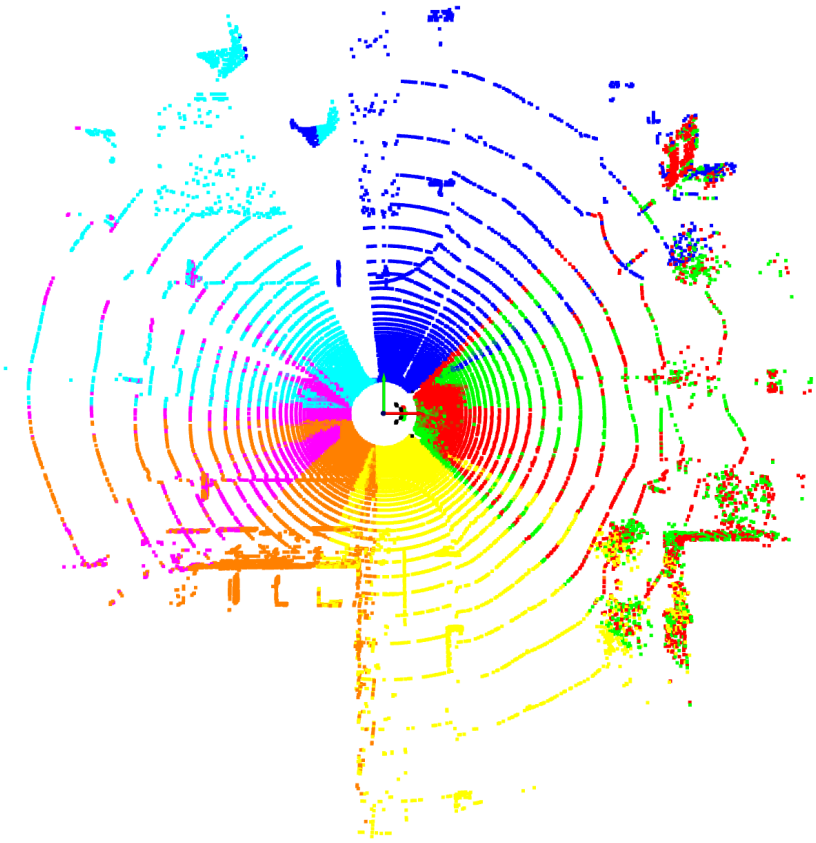

Figure 3: CAMERA_COLORING=True, USE_OCCLUSION=True

In contrast to Figure 1, Figure 3 uses occlusion depth checks to assign each LiDAR point to the RGBD camera with the closest decoded depth, hence the variety in camera mapping color rather than solid blocks.

Figure 4: CAMERA_COLORING=False, USE_OCCLUSION=True

When assigning the colors, this can be confirmed as the points above the green axis are now no longer a light blue. Instead, the front left camera's color is used as that depth is more accurate than the back left. This is because to the back left, the LiDAR point should be close, when in reality it is further away as the front left accurately says.

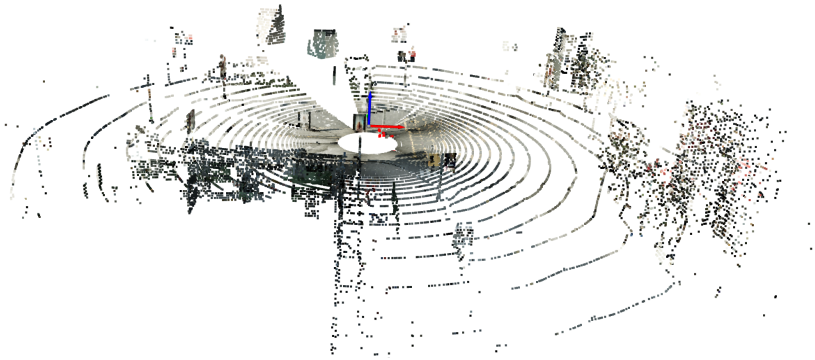

Figure 5: Alternate CAMERA_COLORING=False, USE_OCCLUSION=True

Figure 5 is another angle to see this occlusion check at work, where the building takes on the correct colors and is not assigned the ones from the sign near the axis markers.

5 Bonus

In our notebook, seven cameras are necessary to fill in the collected LiDAR data as stated in Section 2. If a headlight were to be omitted, this number could potentially be six; however, that would violate the constraint on the assignment and therefore we did not test this condition.

6 Accompanying Materials

Based on our understanding of the assignment description, we will be submitting the original two headlight RGBD .avi files, and the LiDAR data per frame in a folder. When running this .ipynb, the data from CARLA will be all seven cameras .avi and corresponding folders for .png files.